

ESPRIT

2025 - 2026

PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

STB SENTINEL

Plateforme proactive de supervision et de gestion
des incidents des systèmes d'information
bancaires

Réalisé par

Malek Jendoubi

Encadrant académique

Monsieur Hamdi Braiek

Organisme d'étude

Société Tunisienne de Banque - contexte académique
simulé

Spécialité

TWIN

Version de rédaction fondée sur l'état réel du dépôt au 20 août 2026

Dédicaces

À mes parents, pour leurs sacrifices, leur patience et leur soutien constant tout au long de mon parcours.

À ma famille, à mes amis et à toutes les personnes qui m'ont encouragé et accompagné dans la réalisation de ce projet.

Je leur dédie humblement ce travail.

Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma gratitude à toutes les personnes qui ont contribué à son aboutissement.

J'adresse mes sincères remerciements à mon encadrant académique, Monsieur Hamdi Braiek, pour ses conseils, sa disponibilité et son accompagnement. Je remercie également le corps professoral et administratif d'ESPRIT pour la qualité de la formation dispensée.

Enfin, je remercie les membres du jury pour l'attention portée à ce travail ainsi que toutes les personnes qui m'ont soutenu durant sa réalisation.

Résumé

La continuité des systèmes d'information constitue un enjeu majeur pour les établissements bancaires. Une indisponibilité, une dégradation progressive ou l'expiration d'un certificat numérique peut affecter les opérations internes et la qualité du service rendu aux clients. Ce projet propose STB Sentinel, une plateforme proactive de supervision et de gestion des incidents conçue dans un contexte académique représentatif de la Société Tunisienne de Banque.

La solution centralise la configuration des systèmes et de leurs points de contrôle, exécute des vérifications HTTP, API JSON et TLS, calcule les états UP, DEGRADED, DOWN et UNKNOWN, puis transforme les anomalies confirmées en alertes dédoublées et en incidents actionnables. Elle couvre également l'affectation aux techniciens, les SLA, les escalades, les maintenances, les notifications internes, e-mail et SMS, ainsi que les indicateurs MTTR, MTTA et MTTR et les exports PDF, Excel et CSV.

La plateforme repose sur Angular, ASP.NET Core .NET 8, Entity Framework Core et PostgreSQL. Quatre microservices indépendants simulent le Core Banking, le RNE, le service SMS et le système RH avec respectivement Oracle, MongoDB, MySQL et SQL Server. Cette approche permet de valider les mécanismes de supervision avant tout raccordement aux SI réels.

Mots-clés : supervision, systèmes bancaires, disponibilité, TLS, alertes, incidents, SLA, Angular, .NET 8, PostgreSQL, microservices, DevOps.

Table des matières

Table des matières	5
Liste des abréviations.....	7
Introduction générale	8
Introduction.....	9
1.1 Présentation du contexte d'accueil	9
1.2 Contexte et problématique.....	9
1.3 Objectifs	9
1.4 Étude de l'existant	9
1.5 Solution proposée	10
Conclusion	10
Introduction.....	11
2.1 Identification des acteurs.....	11
2.2 Besoins fonctionnels	11
2.3 Besoins non fonctionnels	11
2.4 Répartition des privilèges.....	12
2.5 Méthodologie Scrum.....	12
2.6 User Stories structurantes.....	13
2.7 Definition of Done	13
Conclusion	13
Introduction.....	14
3.1 Architecture générale	14
3.2 Choix architectural.....	14
3.3 Communication	15
3.4 Modèle métier.....	15
3.5 Conception du monitoring.....	16
3.6 Conception TLS.....	17
3.7 Cycle d'incident	17
Conclusion	18
Introduction.....	19
4.1 Identité, authentification et audit	19
4.2 Catalogue des SI et environnements	19
4.3 Configuration des endpoints.....	19
4.4 Exécution des contrôles.....	20

4.4.1 Contrôle HTTP	20
4.4.2 Contrôle API JSON	20
4.4.3 Contrôle TLS	20
4.5 Alertes	20
4.6 Incidents	21
4.7 SLA et escalade	21
4.8 Notifications	21
4.9 Maintenance	21
4.10 Dashboard et rapports	22
4.11 Interface Angular	22
4.12 Limites et travaux en cours.....	22
Conclusion	23
Introduction.....	24
5.1 Tests automatisés	24
5.2 Conteneurisation	24
5.3 Cible CI/CD et Kubernetes.....	25
5.4 Observabilité.....	25
5.5 Sécurité de déploiement	25
Conclusion	26
Conclusion générale et perspectives	27
Bilan.....	27
Difficultés rencontrées	27
Perspectives.....	27
Bibliographie et webographie	28
Annexe A - Commandes principales	28
Annexe B - Scénario de démonstration	28

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Delivery
CRUD	Create, Read, Update, Delete
DNS	Domain Name System
HTTP	HyperText Transfer Protocol
HTTPS	HTTP Secure
JWT	JSON Web Token
KPI	Key Performance Indicator
MTTA	Mean Time To Acknowledge
MTTD	Mean Time To Detect
MTTR	Mean Time To Resolve
RBAC	Role-Based Access Control
REST	Representational State Transfer
SI	Système d'information
SLA	Service Level Agreement
TLS	Transport Layer Security
UML	Unified Modeling Language

Introduction générale

La transformation numérique du secteur bancaire augmente la dépendance aux applications, aux API et aux infrastructures interconnectées. La défaillance d'un seul composant peut perturber une chaîne de traitement, retarder une opération ou dégrader l'expérience des collaborateurs et des clients.

Une supervision efficace ne consiste pas seulement à vérifier qu'un serveur répond. Elle doit mesurer la disponibilité, la latence, la conformité des réponses métier et la sécurité des échanges HTTPS. Elle doit également transformer les observations techniques en alertes pertinentes, puis en incidents suivis par les équipes selon des responsabilités et des délais clairement établis.

STB Sentinel répond à cette problématique au moyen d'une plateforme centrale de monitoring actif. En l'absence d'accès aux SI réels durant la phase académique, quatre simulateurs reproduisent des comportements représentatifs et permettent d'injecter des anomalies contrôlées. Le présent rapport expose le contexte, l'analyse des besoins, la méthodologie Scrum, la conception, la réalisation et la trajectoire DevOps de la solution.

1 Cadre général du projet

Introduction

Ce chapitre présente le contexte bancaire étudié, la problématique de continuité des SI, les limites d'une surveillance fragmentée et la solution proposée.

1.1 Présentation du contexte d'accueil

La Société Tunisienne de Banque évolue dans un environnement où les applications métier, les services de communication, les référentiels externes et les systèmes de ressources humaines participent quotidiennement aux opérations. Dans ce rapport, aucune donnée ni infrastructure réelle de la banque n'est exposée : les noms fonctionnels servent à construire un laboratoire académique contrôlé.

1.2 Contexte et problématique

La multiplication des dépendances applicatives rend difficile une détection rapide des pannes. Un service peut répondre tout en étant incapable d'accéder à sa base de données ; un endpoint peut rester disponible mais devenir trop lent ; un certificat TLS peut expirer et interrompre brutalement les échanges HTTPS. Sans centralisation, la détection repose souvent sur des vérifications manuelles ou sur les réclamations des utilisateurs.

1.3 Objectifs

- centraliser le catalogue des SI et de leurs environnements
- exécuter des contrôles HTTP, API JSON et TLS configurables
- conserver une preuve technique de chaque contrôle
- dédupliquer les anomalies et corrélérer les alertes
- piloter le cycle complet des incidents et des SLA
- notifier les acteurs appropriés
- produire des tableaux de bord et rapports exploitables
- préparer un déploiement reproductible et observable

1.4 Étude de l'existant

Tableau 1.1 - Comparaison des approches de supervision

Approche	Atouts	Limites dans le contexte
Vérification manuelle	Simple au démarrage	Non continue, non historisée, dépendante d'une personne

Approche	Atouts	Limites dans le contexte
Logs applicatifs isolés	Détail technique riche	Vision fragmentée et réaction tardive
Outil générique de disponibilité	Mesures et tableaux de bord	Peu de gestion métier des incidents sans intégration
STB Sentinel	Contrôle actif, preuve, alerte, incident et SLA	Nécessite des endpoints, un accès réseau et une configuration fiable

1.5 Solution proposée

STB Sentinel adopte une plateforme centrale modulaire. Elle interroge les endpoints depuis l'extérieur des SI, mesure leur comportement et applique des règles de santé homogènes. Les simulateurs sont indépendants de la plateforme et reproduisent les technologies de données retenues pour les quatre écosystèmes.

Architecture générale de STB Sentinel

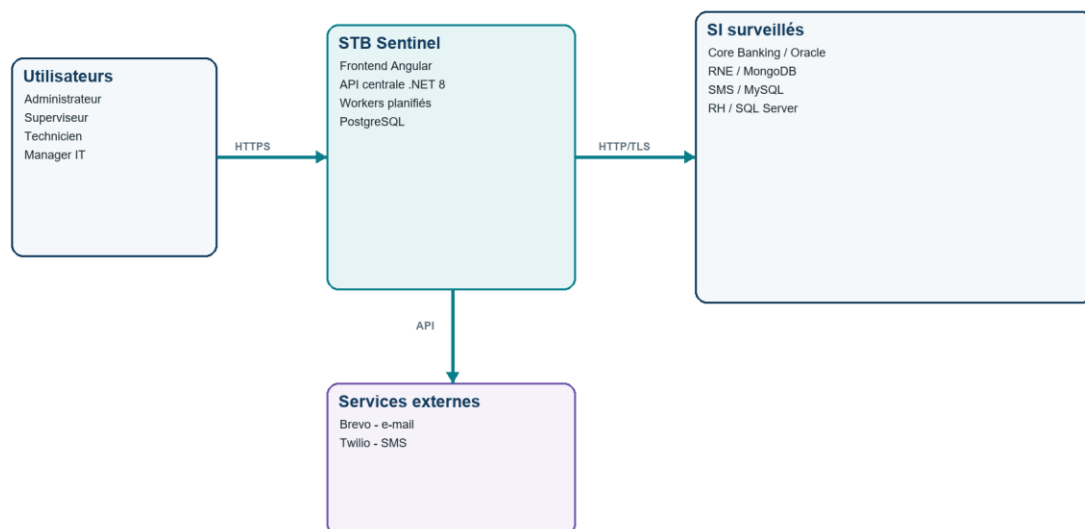


Figure 1.1 - Architecture générale de STB Sentinel

Conclusion

La solution vise à réduire le délai de détection et à structurer la réaction opérationnelle, tout en restant adaptable à de futurs SI réels.

2 Analyse des besoins et méthodologie

Introduction

Ce chapitre identifie les acteurs, formalise les besoins fonctionnels et non fonctionnels, puis organise le projet selon Scrum.

2.1 Identification des acteurs

Tableau 2.1 - Acteurs de STB Sentinel

Acteur	Responsabilité
Administrateur	Comptes, catalogue SI, endpoints, règles et configuration technique
Superviseur	Alertes, qualification, création et affectation des incidents
Technicien	Diagnostic et résolution des incidents qui lui sont affectés
Manager IT	KPI, SLA, rapports, risques et validation des décisions critiques
SI surveillé	Expose les URLs HTTP/HTTPS et certificats contrôlés
Fournisseur externe	Envoie les e-mails ou SMS demandés par la plateforme

2.2 Besoins fonctionnels

Tableau 2.2 - Synthèse des besoins fonctionnels

Gestion	Besoins couverts
Identité	Connexion JWT, profil, comptes actifs, rôles fixes et audit
Catalogue	SI, environnement, criticité, propriétaire et archivage
Endpoints	URL, type, fréquence, timeout, seuils, code et JSON attendus
Supervision	Planification, exécution manuelle, état et historique
TLS	Chaîne, dates, hôte, confiance, négociation et preuve
Alertes	Règles, confirmation, déduplication, acquittement et résolution
Incidents	Corrélation, affectation, timeline, résolution et pièces jointes
SLA	Échéances, AtRisk, Breached, Met et escalade
Maintenance	Calendrier jour/semaine/mois et suspension d'alertes
Reporting	KPI, filtres, vues, PDF, Excel et CSV

2.3 Besoins non fonctionnels

- Sécurité : authentification, autorisation côté API, mots de passe hachés et secrets non exposés.
- Fiabilité : annulation, timeouts, verrou distribué et conservation des preuves.

- Performance : contrôles planifiés indépendants, pagination et index PostgreSQL.
- Disponibilité : endpoints liveness/readiness et services conteneurisables.
- Traçabilité : audit, historique d'incident, horodatages et résultats d'envoi.
- Maintenabilité : séparation Domain, Application, Infrastructure et API.
- Ergonomie : interface responsive, états lisibles et messages orientés action.

2.4 Répartition des privilèges

Tableau 2.3 - Matrice cible des responsabilités

Gestion	Admin	Superviseur	Technicien	Manager IT
Utilisateurs	Gérer	Lire	-	Lire
SI / endpoints	Gérer	Lire et tester	Lire le périmètre	Lire
Alertes	Lire	Acquitter et qualifier	Liées aux incidents	Lire les critiques
Incidents	Lire	Créer/affecter/piloter	Traiter les siens	Valider les critiques
SLA / rapports	Configurer techniquement	Suivre	Consulter le sien	Piloter et valider
Maintenance	Administrer	Planifier	Consulter	Approuver production

2.5 Méthodologie Scrum

Le projet est découpé en incréments courts. Chaque sprint possède un objectif, des User Stories, des critères d'acceptation et une démonstration. Le Product Backlog est priorisé par valeur métier et risque technique.

Tableau 2.4 - Planification synthétique des sprints

Sprint	Objectif
Sprint 0	Cadrage et environnement
Sprint 1	Identité et accès
Sprint 2	Catalogue SI et endpoints
Sprint 3	Monitoring HTTP/API/TLS
Sprint 4	Simulateurs et bases indépendantes
Sprint 5	Alertes et incidents
Sprint 6	Notifications, SLA et escalades
Sprint 7	Maintenance, dashboard et rapports
Sprint 8	Sécurité et qualité
Sprint 9	DevOps et observabilité
Sprint 10	Stabilisation et soutenance

2.6 User Stories structurantes

Tableau 2.5 - User Stories principales

ID	Acteur	Besoin
US-101	Utilisateur	Se connecter et gérer son profil
US-201	Administrateur	Créer un SI et ses endpoints
US-301	Superviseur	Lancer un contrôle manuel
US-303	Plateforme	Détecter disponibilité, latence et certificat
US-501	Plateforme	Dédupliquer les alertes
US-502	Superviseur	Qualifier et affecter un incident
US-503	Technicien	Documenter et résoudre son incident
US-601	Utilisateur	Recevoir des notifications internes, e-mail et SMS
US-701	Manager IT	Consulter santé, SLA et rapports

2.7 Definition of Done

- code compilé et relu
- tests adaptés au risque
- autorisation API et visibilité frontend cohérentes
- migration fournie si le modèle change
- documentation mise à jour
- démonstration reproductible
- aucun secret commis

Conclusion

Le découpage Scrum rend la progression démontrable et permet de traiter en priorité la chaîne de valeur allant de la détection à la résolution.

3 Conception

Introduction

Ce chapitre transforme les besoins en architecture, modèles métier, relations de données et scénarios d'interaction.

3.1 Architecture générale

La solution sépare l'interface Angular, l'API centrale, les workers et les adaptateurs d'infrastructure. Les SI simulés restent externes au cœur de supervision et sont interrogés via HTTP/HTTPS.

Architecture logique en couches

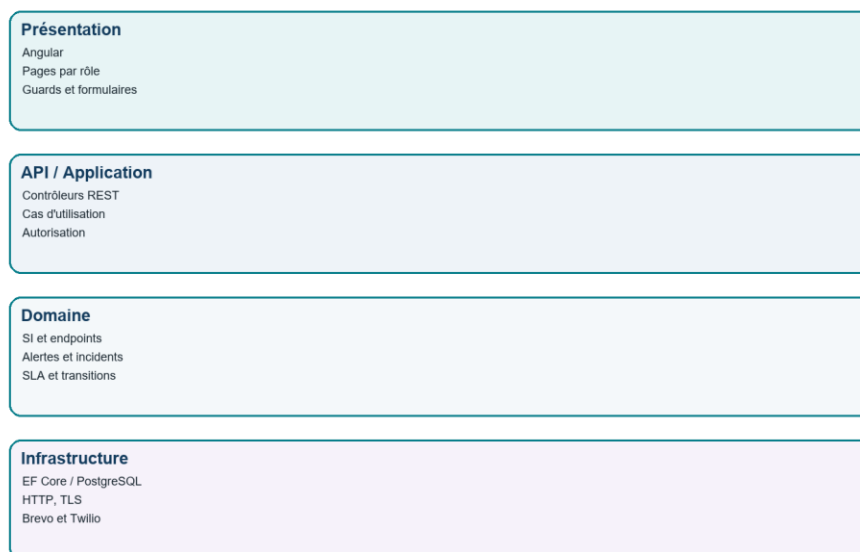
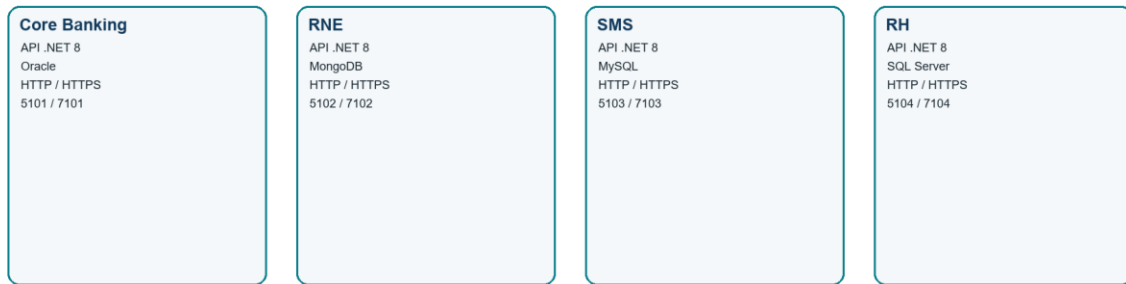


Figure 3.1 - Architecture logique en couches

3.2 Choix architectural

Le backend STB Sentinel est un monolithe modulaire en couches, et non un ensemble de microservices internes. Ce choix limite la complexité transactionnelle tout en maintenant une séparation nette des responsabilités. Les quatre SI simulés sont, eux, des services déployables indépendamment avec leurs propres bases.

Écosystème de simulation des SI



Chaque simulateur est démarrable, testable et conteneurisable indépendamment.

Figure 3.2 - Microservices simulant les SI

3.3 Communication

HTTP/HTTPS constitue le mécanisme principal car le monitoring doit observer le système depuis l'extérieur. RabbitMQ n'est pas obligatoire pour qualifier une architecture distribuée et ne remplace pas le polling lorsqu'un SI est totalement indisponible. Il est conservé comme évolution possible pour des événements asynchrones tels que AlertCreated ou IncidentResolved.

3.4 Modèle métier

Agrégats métier et relations principales

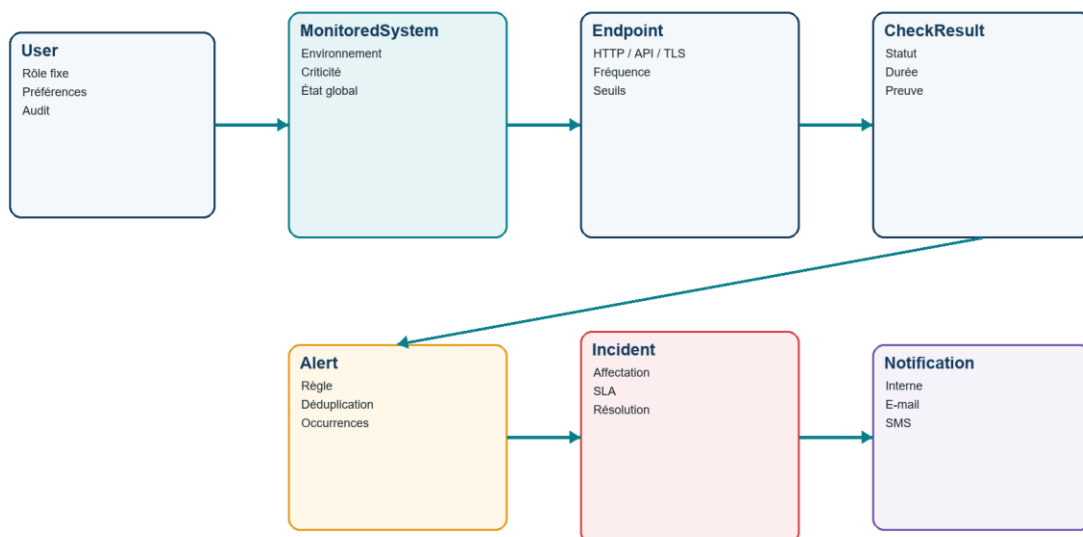


Figure 3.3 - Agrégats métier et relations principales

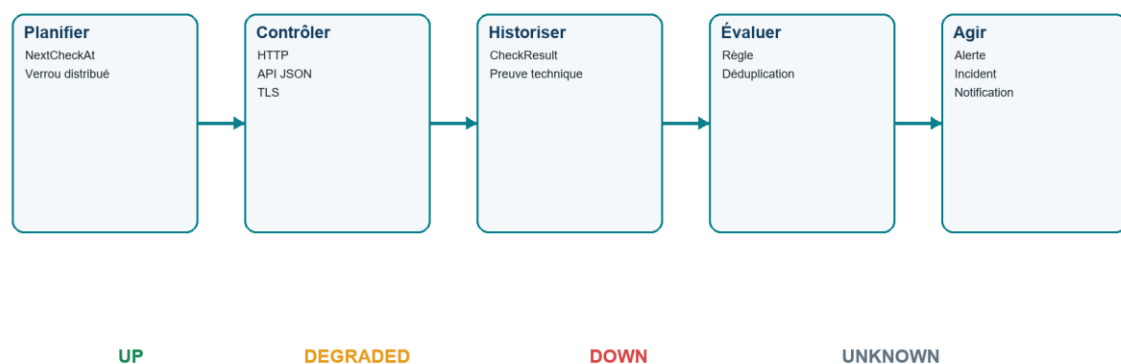
Le modèle persistant comprend actuellement dix-huit tables métier. Les énumérations sont stockées sous forme de texte ; le rôle est un champ de User et aucune table roles, permissions ou user_roles n'est nécessaire.

Tableau 3.1 - Organisation conceptuelle des données

Table ou groupe	Rôle
users	Comptes et rôles
monitored_systems	SI et état agrégé
monitoring_endpoints	Configuration des contrôles
check_results	Historique et preuves
alert_rules	Règles dynamiques
alerts / alert_occurrences	Anomalies dédoublées
incidents	Cycle de traitement
incident_*	Commentaires, histoire, preuves et escalade
sla_policies	Objectifs de service
notifications	Flux interne
maintenance windows	Périodes planifiées
saved_views	Filtres utilisateurs

3.5 Conception du monitoring

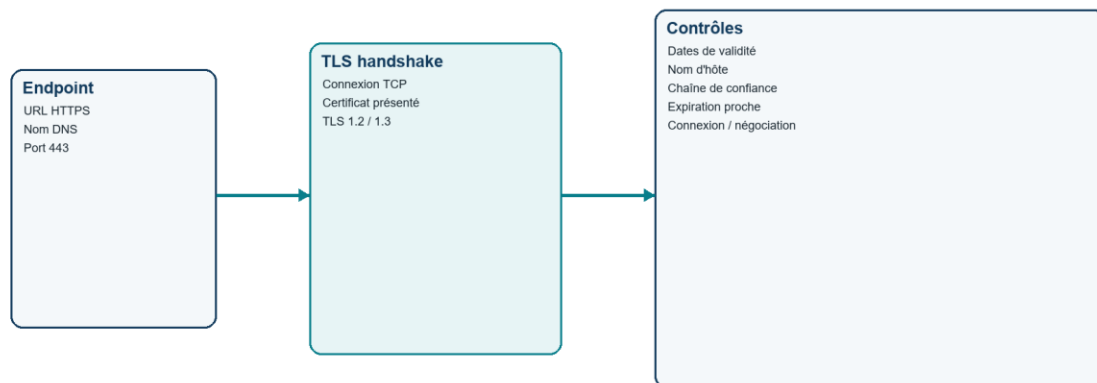
Chaîne active de supervision et de réaction

**Figure 3.4 - Chaîne de supervision**

L'ordre d'enregistrement est important : le résultat technique est persisté avant l'évaluation des alertes, ce qui garantit qu'une alerte référence une preuve déjà disponible.

3.6 Conception TLS

Validation réelle d'un certificat TLS



La preuve enregistrée provient du certificat réellement présenté par le serveur.

Figure 3.5 - Processus de validation TLS

Le contrôle TLS ne dépend pas de données de certificat saisies par l'administrateur. À chaque exécution, le moteur ouvre une connexion réelle, récupère le certificat présenté, construit la chaîne et enregistre les métadonnées. En production, la CA interne de la STB devra être installée dans le magasin de confiance du serveur ou du conteneur.

3.7 Cycle d'incident

Cycle de vie opérationnel d'un incident



Superviseur : qualification et affectation

Validation et clôture

Technicien : diagnostic et résolution

Figure 3.6 - Machine d'état d'un incident

Conclusion

La conception isole les règles métier des dépendances techniques et prépare le remplacement des simulateurs par des URLs réelles sans recompilation.

4 Réalisation

Introduction

Ce chapitre décrit les fonctionnalités effectivement présentes dans le dépôt et explique leur contribution à la chaîne métier.

4.1 Identité, authentification et audit

L'API authentifie l'utilisateur à partir de son nom ou de son e-mail, vérifie le mot de passe haché et l'état du compte, puis génère un JWT contenant le rôle. Les tentatives réussies ou échouées sont auditées. La logique empêche également la désactivation ou le changement de rôle du dernier administrateur actif.

4.2 Catalogue des SI et environnements

Un SI est défini par son code, son nom, sa description, son environnement, sa criticité et son propriétaire. Il peut être désactivé ou archivé. L'archivage est privilégié à la suppression lorsqu'un historique existe, afin de préserver les preuves et rapports.

Tableau 4.1 - Environnements gérés

Environnement	Usage
Production	Système exploité et critique
Preproduction	Validation avant mise en production
Recette	Tests fonctionnels
Development	Développement et laboratoire

4.3 Configuration des endpoints

Tableau 4.2 - Champs principaux d'un endpoint

Champ	Interprétation
URL	Adresse absolue HTTP/HTTPS
CheckType	Http, ApiJson ou Tls
HttpMethod	GET, HEAD ou POST
ExpectedStatusCode	Code attendu, généralement 200
TimeoutSeconds	Durée maximale d'attente
IntervalSeconds	Période entre deux contrôles
DegradedThresholdMs	Seuil de latence dégradée
DownThresholdMs	Seuil critique

Champ	Interprétation
IsCritical	Impact sur l'état global du SI
ExpectedJsonProperty / Value	Validation métier de la réponse

4.4 Exécution des contrôles

MonitoringWorker interroge les endpoints arrivés à échéance. Un verrou consultatif PostgreSQL empêche plusieurs instances de traiter simultanément le même cycle. Chaque endpoint choisit un exécuteur spécialisé et l'échec d'un contrôle n'empêche pas la poursuite des autres contrôles.

4.4.1 Contrôle HTTP

Le moteur vérifie l'accessibilité, le code de réponse, le timeout et la latence. Une réponse correcte peut donc être classée DEGRADED ou DOWN si elle dépasse les seuils configurés.

4.4.2 Contrôle API JSON

Le contrôle API étend HTTP par la lecture d'une propriété JSON. Il permet par exemple de vérifier `database.available=true`. Cette stratégie détecte une base indisponible uniquement si l'endpoint contrôlé dépend réellement de cette base ou expose explicitement son état.

4.4.3 Contrôle TLS

TlsCheckExecutor utilise TcpClient et SslStream pour obtenir le certificat réel. Il distingue expiration, date future, nom incorrect, chaîne non approuvée, expiration proche, connexion impossible et négociation impossible. La chaîne complète est sérialisée dans les métadonnées. La révocation en ligne CRL/OCSP reste à activer pour une cible bancaire de production.

Tableau 4.3 - Classification des erreurs TLS

Code	Signification	État
TLS_EXPIRED	Certificat expiré	DOWN
TLS_EXPIRING	Moins de 30 jours	DEGRADED ou DOWN
TLS_NAME_MISMATCH	Nom DNS absent du certificat	DOWN
TLS_UNTRUSTED	Chaîne non approuvée	DOWN
TLS_HANDSHAKE	Négociation impossible	DOWN
TLS_CONNECTION	Connexion impossible	DOWN

4.5 Alertes

Les règles d'alerte sont des entités persistées et administrables. Le moteur attend le nombre d'échecs configuré, construit une clé de déduplication et réutilise l'alerte active pendant la fenêtre définie. Chaque répétition reste liée à un CheckResult.

Tableau 4.4 - Règles initiales

Règle standard	Échecs	Déduplication
Endpoint indisponible	2	15 min
Performance dégradée	2	15 min
Timeout réseau	1	15 min
Réponse API invalide	1	15 min
Certificat bientôt expiré	1	24 h
Certificat expiré	1	24 h
Certificat invalide	1	60 min

4.6 Incidents

Le superviseur peut créer un incident depuis une alerte. Une recherche sur le même SI et une fenêtre récente permet de rattacher plusieurs alertes à une panne commune. Le technicien documente le résumé, la cause racine, l'action corrective, l'action préventive et la preuve de résolution.

4.7 SLA et escalade

Tableau 4.5 - Politiques SLA initiales

Priorité	Réponse	Résolution
P1 Critical	15 min	60 min
P2 High	30 min	240 min
P3 Medium	120 min	480 min
P4 Low	480 min	1440 min

Un worker recalcule les états OnTrack, AtRisk, Breached et Met. Pour les P1, l'escalade persistée notifie successivement le superviseur puis le manager IT si l'incident reste ouvert. La persistance permet de reprendre le traitement après un redémarrage.

4.8 Notifications

Les notifications internes sont stockées par utilisateur. ApiNotificationChannel utilise Brevo pour l'e-mail et Twilio pour le SMS, avec un mode Simulation qui permet de tester le contrat sans consommer de quota ni stocker de secrets réels.

4.9 Maintenance

Le calendrier propose les vues jour, semaine et mois, ainsi que création, modification, annulation et suppression. Une fenêtre peut conserver les contrôles tout en supprimant temporairement la création d'alertes. Les maintenances annulées sont exclues du calendrier actif.

4.10 Dashboard et rapports

Le dashboard agrège santé des SI, disponibilité, alertes, incidents, SLA et tendances. Les mesures MTDD, MTTA et MTTR représentent respectivement les délais moyens de détection, d'acquittement et de résolution. Les résultats sont exportables en CSV, PDF et Excel .xlsx sur des périodes mensuelles ou annuelles.

Tableau 4.6 - Indicateurs opérationnels

Indicateur	Interprétation
MTDD	Temps entre début estimé et détection
MTTA	Temps entre alerte et acquittement
MTTR	Temps entre création et résolution
Disponibilité	Part des contrôles réussis
Conformité SLA	Part des incidents résolus dans le délai

4.11 Interface Angular

Tableau 4.7 - Principaux écrans Angular

Écran	Finalité
Connexion	Authentification et retour vers la route demandée
Dashboard	Synthèse adaptée au rôle
Systèmes	Catalogue et filtres
Détail SI	Endpoints, contrôles et preuves TLS
Alertes	Liste, acquittement et règles
Incidents	Filtres, vues sauvegardées et affectation
Détail incident	Timeline, pièces jointes et résolution
Calendrier	Maintenances jour/semaine/mois
Profil	Identité et préférences de notification

4.12 Limites et travaux en cours

Tableau 4.8 - Écarts entre implémentation et cible

Sujet	État ou action
Privilèges	La matrice cible de séparation stricte doit remplacer les droits encore trop larges de l'Admin.
TLS production	Activer révocation en ligne et éventuellement mTLS.
RabbitMQ	Mentionné comme cible mais absent de l'implémentation actuelle.
DevOps	Docker des simulateurs présent ; Jenkins/Kubernetes/Prometheus/Grafana restent à finaliser.

Sujet	État ou action
Accès SI réels	Nécessite DNS, firewall, endpoints autorisés et CA interne de confiance.

Conclusion

La réalisation couvre le cœur fonctionnel de la chaîne de supervision et rend chaque résultat traçable. Les travaux restants concernent principalement le durcissement des rôles, la production TLS et l'industrialisation.

5 DevOps, tests et déploiement

Introduction

Ce chapitre présente les éléments disponibles dans le dépôt et la cible d'industrialisation retenue.

5.1 Tests automatisés

Tableau 5.1 - Organisation des tests

Famille	Couverture
Unitaires	User, sprints 1-3, maintenance et cycle incident
Architecture	Respect des dépendances entre couches
Autorisation	Réponses 401/403 et politiques
Intégration PostgreSQL	Migrations, persistance et cycle métier

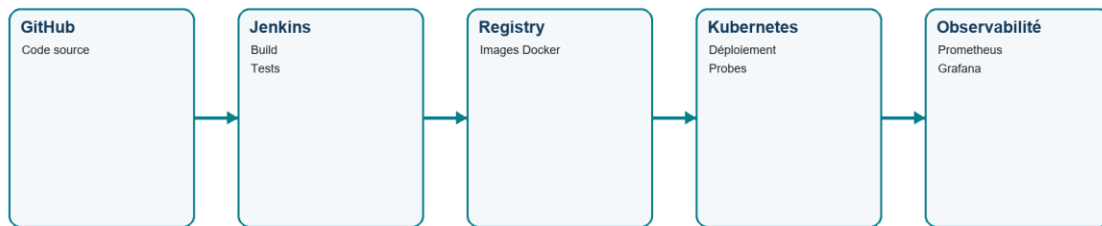
La solution compile actuellement sans erreur ni avertissement. Les tests doivent évoluer avec la nouvelle matrice des privilèges afin de vérifier que chaque rôle ne peut exécuter que ses responsabilités.

5.2 Conteneurisation

Chaque simulateur possède un Dockerfile et un fichier Docker Compose associé à sa base. La base PostgreSQL centrale est également démarrable par Compose. Cette organisation permet d'arrêter une dépendance isolément et d'observer l'impact depuis STB Sentinel.

5.3 Cible CI/CD et Kubernetes

Chaîne DevOps cible



Cible d'industrialisation - les manifestes Kubernetes et pipelines restent à finaliser.

Figure 5.1 - Chaîne DevOps cible

La cible prévoit une image par composant, des manifestes Deployment, Service, ConfigMap, Secret et PVC, des probes liveness/readiness et des rolling updates. Jenkins doit enchaîner compilation, tests, analyse, construction des images, publication et smoke tests. Ces éléments sont décrits comme perspectives tant que leurs fichiers ne sont pas présents dans le dépôt.

5.4 Observabilité

Prometheus et Grafana doivent compléter la supervision fonctionnelle par des métriques techniques portant sur l'API, les workers, les simulateurs et les pods. Les métriques recommandées comprennent le nombre et la durée des contrôles, les erreurs, les alertes créées, les notifications échouées et l'état des exécutions planifiées.

5.5 Sécurité de déploiement

- injection des secrets par variables protégées ou Kubernetes Secrets
- bases non exposées publiquement
- CA interne montée dans les conteneurs de confiance
- images versionnées par SHA Git
- probes et limites de ressources
- sauvegarde et procédure de rollback
- aucune donnée réelle dans le laboratoire académique

Conclusion

La structure actuelle prépare l'industrialisation, mais la chaîne DevOps complète doit être finalisée et démontrée avant de la présenter comme acquise.

Conclusion générale et perspectives

STB Sentinel démontre qu'une plateforme centrale peut couvrir une chaîne cohérente allant du contrôle technique à la gestion opérationnelle. L'utilisation d'endpoints configurables permet de surveiller disponibilité, performance, contenu métier et certificats sans lier le moteur à un SI particulier.

Le recours à quatre simulateurs indépendants a permis d'éprouver le comportement de la plateforme en l'absence d'accès aux systèmes réels. Cette séparation reste fidèle à une intégration future : le passage en environnement STB reposera principalement sur le remplacement des URLs, l'ouverture des flux réseau, la gestion des authentifications et l'installation des autorités de certification internes.

Les perspectives prioritaires sont le durcissement de la séparation des privilèges, la validation de révocation TLS, la prise en charge éventuelle de mTLS, l'industrialisation Kubernetes/Jenkins, l'observabilité Prometheus/Grafana et l'ajout ultérieur d'une détection prédictive explicable.

Bilan

Le projet a permis de mettre en œuvre une architecture en couches, un moteur de monitoring actif, des règles dynamiques, une gestion d'incidents structurée et une stratégie de test couvrant domaine, autorisation et persistance.

Difficultés rencontrées

- modéliser des SI non accessibles réellement
- distinguer disponibilité d'API et santé de base de données
- valider TLS sans accepter silencieusement un certificat incorrect
- éviter les doubles exécutions et tempêtes d'alertes
- définir une séparation cohérente entre rôles techniques et décisionnels

Perspectives

- connexion contrôlée aux SI réels
- révocation TLS et mTLS
- service d'événements asynchrones si un besoin RabbitMQ est confirmé
- déploiement Kubernetes reproductible
- alerting technique Prometheus/Grafana
- modèles d'anomalie explicables avec validation humaine

Bibliographie et webographie

1. Schwaber, K. et Sutherland, J. The Scrum Guide, version 2020. <https://scrumguides.org/scrum-guide.html>
2. Microsoft. Documentation ASP.NET Core et .NET 8. <https://learn.microsoft.com/aspnet/core/>
3. Angular. Documentation officielle. <https://angular.dev/docs>
4. PostgreSQL Global Development Group. Documentation PostgreSQL. <https://www.postgresql.org/docs/>
5. Kubernetes. Concepts et documentation officielle. <https://kubernetes.io/docs/concepts/>
6. OWASP Foundation. Application Security Verification Standard. <https://owasp.org/www-project-application-security-verification-standard/>
7. IETF. RFC 8446 - The Transport Layer Security (TLS) Protocol Version 1.3. <https://www.rfc-editor.org/rfc/rfc8446>

Annexe A - Commandes principales

Tableau A.1 - Commandes de démarrage et test

Composant	Commande
Backend	<code>dotnet run --project platform/backend/StbMonitoring.Api/StbMonitoring.Api.csproj</code>
Frontend	<code>npm start depuis platform/frontend</code>
Tests	<code>dotnet test StbMonitoring.sln</code>
RNE	<code>docker compose -f simulators/docker-compose.rne.yml up --build</code>
SMS	<code>docker compose -f simulators/docker-compose.sms.yml up --build</code>
RH	<code>docker compose -f simulators/docker-compose.hr.yml up --build</code>
Core Banking	<code>docker compose -f simulators/docker-compose.core-banking.yml up --build</code>

Annexe B - Scénario de démonstration

1. Démarrer PostgreSQL, l'API, Angular et un simulateur.
2. Ajouter ou vérifier le SI et ses endpoints HTTP/API/TLS.
3. Exécuter un contrôle et constater l'état UP.
4. Arrêter la base du simulateur ou activer une anomalie.
5. Attendre les échecs consécutifs et observer l'alerte dédoublée.
6. Acquitter l'alerte, créer l'incident et l'affecter.
7. Documenter la résolution et relancer le contrôle.
8. Vérifier le retour UP, la résolution et le rapport.